

EVENT MANAGEMENT SYSTEM

Functional, Non-Functional Requirements and Requirement Analysis

1. Functional and Non-Functional Requirements

1.1 Functional Requirements

Req ID	Type	Description	Priority	Acceptance Criteria	Rationale / Comments
FR-01	Functional	When someone signs up, they choose whether they're an Organizer or an Attendee, and that choice decides what their dashboard looks like from then on.	High	Pass: log in as an Organizer and you see the event-creation tools; log in as an Attendee and you see your tickets and browsing options instead. Fail: an Attendee can reach the event-creation screen, or can see another user's ticket history.	This is the split that everything else depends on — keeping people who host events and people who attend them on separate rails, with separate data.
FR-02	Functional	Organizers build an event by filling in the basics: title, date, time, venue, how many tickets are available, and the price. They can edit it and then publish it when it's ready.	High	Pass: as soon as an event is published, it shows up on the public listings page. Fail: an event goes live without a capacity limit set, or it's published but doesn't actually appear anywhere public.	Nothing else in the system works until an event exists to sell tickets against, so this is really the starting point of the whole flow.
FR-03	Functional	Attendees can look through active events, narrow the list down by date or category, and open a page to see the full details.	Medium	Pass: searches come back with accurate, currently-active events in under 2 seconds. Fail: events that have already happened or been canceled still show up in the default search results.	If people can't find events easily, sales and attendance both suffer — discovery is what drives the rest of the funnel.
FR-04	Functional	The moment someone successfully registers, the number of tickets left for that event drops right away.	High	Pass: buying 2 tickets lowers the remaining count by 2, and if only 1 ticket is left, trying to buy 2 gets blocked. Fail: the system lets more tickets sell than the venue actually holds.	This is basically a safety issue as much as a business one — overselling a venue isn't just embarrassing, it can be a legal problem.
FR-05	Functional	Payments go through a secure external payment gateway, and once a charge is approved, a unique digital ticket is created for the buyer.	High	Pass: an approved payment immediately produces a Ticket ID and sends out a confirmation email. Fail: the card gets charged but no ticket is ever generated, or the inventory count doesn't move.	Money changing hands has to line up exactly with a ticket being issued — any gap there turns into a support headache or worse.

Req ID	Type	Description	Priority	Acceptance Criteria	Rationale / Comments
FR-06	Functional	Attendees can cancel a registration before the event happens, though whatever gets refunded depends on the Organizer's own refund policy.	Medium	Pass: canceling voids the Ticket ID and puts that seat back into available capacity. Fail: a canceled ticket is still being counted as an occupied seat.	Gives attendees an easy way out and lets Organizers actually resell a seat instead of it sitting empty for no reason.
FR-07	Functional	Organizers can pull a final list of everyone who's attending — names, emails, and ticket IDs — and download it.	Medium	Pass: the download is a clean CSV that only includes people who actually paid and are confirmed. Fail: canceled tickets sneak into the list looking like they're still active.	Staff need this on the day of the event for check-in, and Organizers reuse it afterward for follow-up marketing.
FR-08	Functional	Platform Admins have the ability to suspend an event or an entire Organizer account if it breaks the terms of service.	High	Pass: suspending an event pulls it from public listings and stops any new sales right away. Fail: someone can still check out on a suspended event by hitting the link directly.	This is the platform's last line of defense against fraud and policy violations, so it needs to actually work instantly, not eventually.

1.2 Non-Functional Requirements

Req ID	Type	Description	Priority	Acceptance Criteria	Rationale / Comments
NFR-01	Non-Functional (Performance)	Checkout needs to hold up under load — up to 10,000 people trying to buy at once — without pages taking more than 3 seconds to load.	High	Pass: load tests show sub-3-second responses even during a simulated flash-sale rush. Fail: the database chokes or the server times out once traffic spikes.	Popular events cause sudden traffic surges, and a checkout that falls over during a sale costs both money and reputation.
NFR-02	Non-Functional (Security)	Full credit card numbers are never stored on our end — all payment data gets tokenized so we stay within PCI-DSS rules.	High	Pass: a security audit finds only transaction tokens and masked card numbers (last 4 digits) in the database. Fail: a raw card number turns up anywhere in logs or storage.	Storing real card numbers is a liability nobody wants — tokenization keeps us compliant and keeps customer data safer.
NFR-03	Non-Functional (Reliability)	Ticket bookings run through ACID-compliant transactions specifically so two people can't both grab the same last seat.	High	Pass: if two people try to buy the last ticket at the same instant, one purchase goes through and the other gets a clear failure message. Fail: both people end up charged for a single remaining ticket.	Double-booking is the kind of bug that erodes trust fast and usually ends in an awkward manual refund conversation.

Req ID	Type	Description	Priority	Acceptance Criteria	Rationale / Comments
NFR-04	Non-Functional (Usability)	The Attendee side of the site is built mobile-first, so someone can complete an entire purchase comfortably from their phone.	Medium	Pass: layout adapts properly and checkout fields never force horizontal scrolling on a phone screen. Fail: buttons are too small to tap, or the payment form breaks on a mobile viewport.	Most people are buying tickets from their phones these days, so a clunky mobile checkout directly costs conversions.
NFR-05	Non-Functional (Maintainability)	Payment processing is kept loosely coupled from the core inventory logic, talking to it only through a defined API layer.	Low	Pass: switching payment providers (say, Stripe to PayPal) only touches the payment service layer, not the inventory database. Fail: payment logic ends up hardcoded directly into the ticket reservation queries.	Payment vendors change for business reasons more often than people expect, and this keeps that from turning into a rewrite.

2. Requirement Analysis — User, System and Domain

2.1 User Requirements

ID	User Requirement
UR-01	As an Event Organizer, I want to put together a detailed event page so people know exactly what the event is, where it's happening, and what it costs.
UR-02	As an Event Organizer, I want a hard cap on ticket sales so I never accidentally sell more seats than the venue can actually hold.
UR-03	As an Attendee, I want to search and filter events by date or category so I can quickly find things that actually interest me.
UR-04	As an Attendee, I want to pay for my ticket securely with a credit card so my spot is locked in right away.
UR-05	As an Attendee, I want an email receipt with a digital ticket attached so I have something to show at the door.
UR-06	As an Event Organizer, I want to export the list of everyone who bought a ticket so my staff can check people in smoothly at the venue.
UR-07	As a Platform Admin, I want the ability to take down fraudulent events so users don't get scammed on the platform.

2.2 System Requirements

ID	System Requirement
SR-01	Check sessions and apply role-based access control so people land on the right dashboard — Admin, Organizer, or Attendee — as soon as they log in.
SR-02	Lock records at the database level during checkout (whether optimistic or pessimistic locking) so the same inventory unit can't be sold twice at once.
SR-03	Talk to an external payment gateway over RESTful APIs to handle authorizing, capturing, and tokenizing payments.
SR-04	Expose a webhook endpoint that listens for success or failure callbacks from the payment gateway and kicks off ticket generation from there.
SR-05	Hook into a transactional email service (something like SendGrid or AWS SES) to send out PDF tickets and receipts without blocking the checkout flow.
SR-06	Run a query that joins the user and ticket tables together and outputs a downloadable CSV manifest for Organizers.

2.3 Domain Requirements

ID	Domain Requirement
DR-01	Every ticket needs a globally unique identifier (a UUID) plus a scannable barcode or QR code that can be validated at the venue door.
DR-02	Payment handling has to follow PCI-DSS standards, and refunds must always go back to the original payment method — never somewhere else — to avoid money-laundering risk.
DR-03	Attendee data like names and emails has to be handled in line with regional privacy rules such as GDPR or CCPA, including getting explicit opt-in before sending marketing emails.
DR-04	Digital ticket inventory can never exceed what fire code and building regulations allow for a given venue — the software isn't allowed to override real-world legal limits.